

The CTC Algorithm

A study guide, stitched from the manim-concepts road

manim-concepts

Contents

How to read this guide	2
1 Counting rules	3
1.1 Independent stages multiply	3
1.2 Order matters: the shrinking pool	4
1.3 Order does not matter: divide out the overcount	4
1.4 Partitions: the same move, blockwise	4
1.5 Which rule does a problem want?	5
2 The alignment problem, and CTC's answer	8
2.1 The blank token, and the collapse map	8
2.2 A transcript's probability is a sum over paths	9
2.3 How big is the sum?	9
2.4 The forward trellis	9
2.5 Where CTC applies — and where it cannot	10
Bibliography	12

How to read this guide

The objective is full comprehension of the CTC algorithm — the alignment machinery, the loss, the gradient, the training dynamics, and decoding. The road runs through fourteen stations; each is a chapter, and between chapters a short transition marks where you stand:



The order is the dependency order: the CTC chapters pose the problem early, the middle chapters build the blocks it needs, and the closing chapters spend them. Practice problems close every chapter; worked solutions live in the separate solutions manual, keyed by the same problem numbers. Worked numbers are spliced from the repo’s verification anchors — for instance, the guide’s central worked probability $P(\text{AB} \mid X) = 0.4877$ — never retyped.

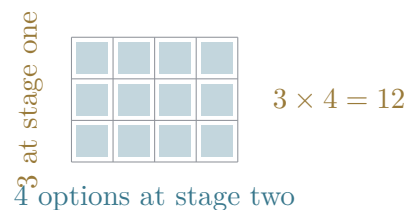
Chapter 1

Counting rules

Every question this book asks eventually reduces to *how many ways can this happen, and how heavy is each way?* The second half of that question is probability's business, and it comes later. This chapter is the first half alone: four counting rules, each built by counting real objects before any formula appears — and a closing observation that the four are secretly one rule wearing different amounts of correction.

1.1 Independent stages multiply

Suppose a choice happens in stages: first one of 3 options, then — whatever was chosen first — one of 4 options. Drawn as a tree, the first choice opens 3 branches and each branch opens 4 more: 12 leaves. Drawn better, the same count is a *grid*: stages become axes, and the total is literally the area of a 3×4 rectangle.



The grid earns the rule — the classical rule of product [4] — its condition: the stages must be *independent* in the counting sense — the *number* of options at stage two cannot depend on what stage one chose (the options themselves may differ; their count may not). When that holds,

$$N = n_1 n_2 \cdots n_k.$$

The formula is the last thing on screen in the video, and the last thing in this section: first the leaves were counted, then the rectangle, and only then the product that summarises what just happened.

1.2 Order matters: the shrinking pool

Fill r slots from a pool of n distinct items, where filling a slot *removes* the item. Three podium places from five runners: the first slot has 5 choices, the second 4, the third 3 — a product again, but of a pool that shrinks: $5 \times 4 \times 3 = 60$. The factorial form is bookkeeping for exactly this: write $5!$ and cancel the tail you never used,

$$P_r^n = \frac{n!}{(n-r)!}, \quad P_3^5 = \frac{5!}{2!} = 60.$$

Hold on to the shrinking pool itself — not just the formula. It is the picture of *sampling without replacement*, and several sections from now it will be the exact thing that breaks independence.

1.3 Order does not matter: divide out the over-count

Choose 3 letters from 5 where the *set* is the answer — $\{A, C, E\}$ chosen in any order is the same choice. The shrinking pool counted 60 ordered selections, but every set of 3 was counted once per ordering: $3! = 6$ times. The correction is a division [1],

$$C_r^n = \binom{n}{r} = \frac{n!}{r!(n-r)!}, \quad \binom{5}{3} = \frac{60}{6} = 10.$$

This is the chapter's load-bearing move — *count with too much order, then divide out the order that changes nothing* — and it is the move the rest of the road keeps reusing.

1.4 Partitions: the same move, blockwise

Split 6 distinct items into labelled groups of sizes 3, 2, 1. Order all 6 ($6!$ ways), chop the ordering into blocks — and notice that reordering *within* a block changes nothing. Divide out each block's internal orderings:

$$N = \frac{n!}{n_1! n_2! \cdots n_k!}, \quad \frac{6!}{3! 2! 1!} = 60.$$

A combination is the special case with two blocks (chosen, not-chosen).

1.5 Which rule does a problem want?

The closing scene of the series maps problem shapes to rules, then closes on an observation worded with care: *multiply the choices — divide only when orderings mean the same outcome*. Three of the four rules are the product rule with unwanted orderings divided out afterwards; the product rule itself divides nothing. Ask two questions of any counting problem — *do the stages' option-counts depend on earlier choices?* (if yes, the pool is shrinking) and *does swapping two items change the answer?* (if no, divide the orderings out) — and the right rule falls out.

Where this goes. The grid returns weighted (areas instead of counts) when probability arrives; the shrinking pool returns as the thing that breaks independence; the binomial coefficient returns as a count of *sorted columns*; and the very next chapter opens a search space of size $3^4 = 81$ raw sequences — the product rule sizing a problem that will take half this book to finish solving.

Practice problems

Problem 1.1. A per-frame labeller writes one of 3 symbols at each of 4 frames, independently. How many distinct label sequences can it write?

Hint: One stage per frame; the option count never changes.

Problem 1.2. Eight runners; how many distinct podiums (gold, silver, bronze) are possible?

Hint: Slots that empty a shrinking pool — and the medals make order matter.

Problem 1.3. From the same eight runners, how many three-person relay *teams* (no roles) can be formed? Check your answer against the previous problem using the overcount argument.

Hint: Every team was counted once per internal ordering.

Problem 1.4. How many distinct arrangements does the word SEDED have?

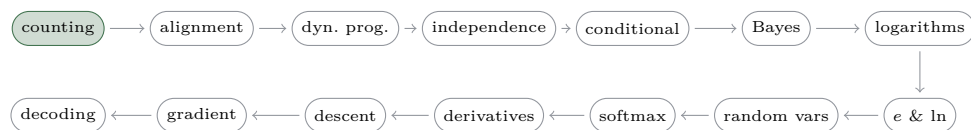
Hint: Six letters, but reordering equal letters changes nothing — divide blockwise.

Problem 1.5. [stretch] A walker on a grid moves from the bottom-left corner to the top-right corner of a 2-row-by-4-column lattice of steps, moving

only right or up — 4 rights and 2 ups in some order. How many routes are there?

Hint: A route is a word in $\{R, U\}$ with repeated letters.

The road so far



You own two moves now: *independent stages multiply*, and *count with too much order, then divide the surplus out*. The last problem left you holding a number — 15 routes through a small lattice — that seemed to be about a walker.

The next chapter poses the problem this whole guide exists to solve, and your two moves are the first tools it reaches for: the product rule sizes a search space of $3^4 = 81$ raw sequences in one line, and the divide-out instinct is what makes the constrained count $\binom{T+U}{T-U}$ readable. The walker’s lattice is about to reappear as a grid of *states against frames* — same fifteen, different costume — and the question will stop being “how many?” and become “how heavy is each one?”. That second question is a debt the chapter takes on openly; most of this book is the repayment.

Chapter 2

The alignment problem, and CTC’s answer

Here is the problem the whole road exists to solve. A speech clip is T audio frames; its transcript is U characters, with $U < T$ and no marking of which frames belong to which character. The transcript says *what* was said, never *when*. Any training signal that demands per-frame labels demands data nobody has.

Connectionist Temporal Classification (CTC) [2] is the standard answer, and this chapter builds its machinery — using, on credit, one notion the road has not yet earned: each frame t carries a score $y_t(k)$ for each symbol k , which we will treat as “the model’s per-frame probability”. What those scores are and why they behave like probabilities is exactly what the middle chapters of this guide construct; here they are simply weights the model hands us. (The video series carries the same declared debt in its Scope.)

2.1 The blank token, and the collapse map

Let the model emit one symbol per frame from the alphabet plus one invented symbol: the *blank* ε , “nothing new to emit”. A raw emission sequence — a *path* — becomes a transcript by the collapse map $\mathcal{B}$: **merge** adjacent repeats, **then drop** blanks. The order is load-bearing: merging first lets a blank protect a genuine double letter (HEL ε LO survives as HELLO), while dropping first would weld the two L’s into one. Without the blank, a per-frame emitter with repeat-merging can never write a double letter at all.

2.2 A transcript’s probability is a sum over paths

Fix the worked example the whole road shares: alphabet $\{A, B, \varepsilon\}$, transcript $Y = AB$, and $T = 4$ frames. Many paths collapse to AB — $AA\varepsilon B$, εABB , $ABB\varepsilon$, $\dots$ — and the data cannot prefer one of them, so the model must not be forced to. CTC’s defining move: score the transcript by *every* path that spells it,

$$P(Y | X) = \sum_{\pi \in \mathcal{B}^{-1}(Y)} \prod_{t=1}^T y_t(\pi_t).$$

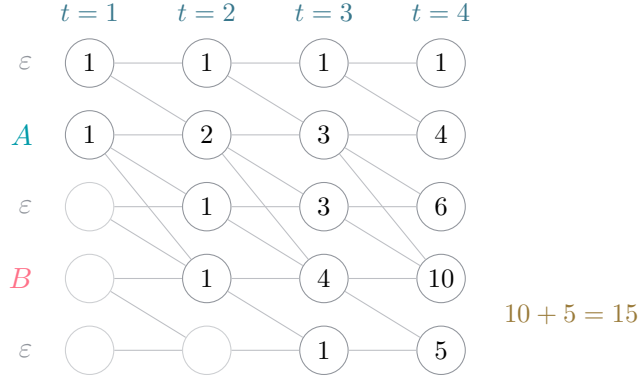
Each path’s weight is a product of per-frame scores (the frames are treated as independent given the input — a real assumption, examined at the chapter’s end), and the transcript’s probability is the sum of those products. Not the best path: the sum. Greedy decoding — take each frame’s favourite symbol and collapse — can return a transcript that no single path dominates but the sum ranks first; the sum is the object CTC trains on.

2.3 How big is the sum?

The product rule from the last chapter sizes the raw space instantly: 3 symbols over 4 frames is $3^4 = 81$ paths. Of those, exactly 15 collapse to AB — countable by hand, and counted on screen in the video. In general the repeat-free count is $\binom{T+U}{T-U}$, and at a realistic $T = 100$, $U = 50$ that is roughly 2×10^{40} — enumeration is dead on arrival. A sum this size must be *reorganised*, not listed.

2.4 The forward trellis

Wrap the transcript in blanks — $AB \rightarrow (\varepsilon, A, \varepsilon, B, \varepsilon)$, giving $2U + 1 = 5$ states — and lay the states against the frames as a grid. A path is a walk down the grid’s legal edges: *stay* in a state, *advance* to the next, or *skip* over a blank — legal only between different letters, because skipping the blank between equal letters would merge them.



Paths sharing a prefix share their count — that is the reorganisation [3]. Let $\alpha_t(s)$ carry the weight of every partial path reaching state s at frame t ; each column needs only the column before it,

$$\alpha_t(s) = (\alpha_{t-1}(s) + \alpha_{t-1}(s-1) + \alpha_{t-1}(s-2)) y_t(z'_s),$$

with the $s-2$ skip term present only where the grid allows it, and the transcript's total read off the two final states. At unit weights the recurrence is pure counting and lands on the same 15 as the hand count — two routes, one answer. Swap the units for the per-frame scores and the same grid computes $P(Y | X)$ exactly: an astronomically wide sum, priced on twenty nodes.

2.5 Where CTC applies — and where it cannot

Three assumptions are the trellis's shape: alignment is *monotonic* (audio does not reorder), the output is *no longer* than the input, and frames are *independent given the input*. Monotonic transduction — speech, handwriting, OCR — fits. Translation reorders, so it does not; that failure belongs to attention models. And the independence assumption leaves language on the table, which is why real systems add an external language model. One reading habit, now, before it can calcify: a trained CTC model's output spikes are *not* segment boundaries — why they look the way they do is this guide's final chapter.

Where this goes. The per-frame scores are debt: the middle chapters build probability from the ground up until softmax and the likelihood lens make y_t honest. The trellis returns twice more — once named for what it is (dynamic programming), and once swept backward, when the gradient chapter differentiates this very sum.

Practice problems

Problem 2.1. Apply the collapse map: what are $\mathcal{B}(\text{AA}\varepsilon\text{B})$, $\mathcal{B}(\varepsilon\text{ABB})$, and $\mathcal{B}(\text{ABBA})$?

Hint: Merge adjacent repeats first, then drop blanks.

Problem 2.2. Show by example that the collapse order matters: find a path that yields a double letter under merge-then-drop but loses it under drop-then-merge.

Hint: Put a blank between two equal letters.

Problem 2.3. Over $T = 3$ frames with alphabet $\{A, \varepsilon\}$, how many raw paths are there, and how many collapse to the transcript “A”?

Hint: The raw space is the product rule; for the collapse count, either enumerate the eight paths or run the unit-weight trellis on $(\varepsilon, A, \varepsilon)$.

Problem 2.4. Using the repeat-free count $\binom{T+U}{T-U}$, how many paths spell a 2-letter transcript with no repeated letter at $T = 5$?

Hint: Substitute and compute; the answer also appears in the gradient chapter’s dominance table.

Problem 2.5. [stretch] The chapter claims greedy decoding can be wrong. Over the two-symbol alphabet $\{A, \varepsilon\}$ at $T = 2$, construct per-frame scores where the frame-wise argmax path collapses to a transcript that is NOT the highest-probability transcript.

Hint: Give the blank a narrow win at every frame, and let the letter’s many paths pool their weight against the blank’s single path.

Bibliography

- [1] *Combination*. <https://en.wikipedia.org/wiki/Combination>. via combinatorics/README.md.
- [2] *Graves et al., 2006 — Connectionist Temporal Classification*. https://www.cs.toronto.edu/~graves/icml_2006.pdf. via deep_learning/README.md.
- [3] *Hannun, "Sequence Modeling with CTC", Distill 2017*. <https://distill.pub/2017/ctc/>. via deep_learning/README.md.
- [4] *Rule of product*. https://en.wikipedia.org/wiki/Rule_of_product. via combinatorics/README.md.